



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное
учреждение
высшего образования
«Московский государственный технический университет имени
Н. Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н. Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Программная инженерия»

Лабораторная работа № 1 **по дисциплине «Анализ алгоритмов»**

Тема Сравнительный анализ ресурсной эффективности алгоритмов

Студент Понарин А. А.

Группа ИУ7-55Б

Преподаватели Волкова Л. Л., Строганов Д. В.

Москва, 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 Аналитическая часть	4
1.1 Метод полного перебора (Линейный поиск)	4
1.2 Рекурсивный бинарный поиск	4
1.3 Итеративный бинарный поиск №1 (с двумя сравнениями)	5
1.4 Итеративный бинарный поиск №2 (с одним сравнением)	5
2 Конструкторская часть	6
2.1 Схема алгоритма полного перебора	6
2.2 Схема алгоритма бинарного поиска с двумя сравнениями	6
2.3 Схема алгоритма бинарного поиска с одним сравнением	7
2.4 Схема алгоритма рекурсивного бинарного поиска	8
2.5 Требования к программному обеспечению	8
3 Технологическая часть	9
3.1 Средства реализации	9
3.2 Реализация алгоритмов	9
4 Модульное тестирование	13
5 Исследовательская часть	16
5.1 Теоретическая оценка затрачиваемой памяти	16
5.2 Оценка вычислительной сложности алгоритмов как функции от числа сравнений	16
5.2.1 Метод полного перебора	16
5.2.2 Итеративный бинарный поиск с двумя сравнениями	16
5.2.3 Итеративный бинарный поиск с одним сравнением	17
5.2.4 Рекурсивный бинарный поиск	17
5.3 Сравнительный анализ алгоритмов	17
ЗАКЛЮЧЕНИЕ	19
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	20

ВВЕДЕНИЕ

Цель работы — выполнение сравнительного анализа ресурсной эффективности алгоритмов поиска заданного элемента в массиве чисел.

Задачи:

1. описать решаемую согласно варианту задачу;
2. разработать рекурсивный и нерекурсивные алгоритмы решения задачи;
3. описать разработанные алгоритмы схемами алгоритмов;
4. описать требования к программному обеспечению;
5. описать средства реализации алгоритмов;
6. реализовать разработанные алгоритмы;
7. выполнить тестирование реализации алгоритмов;
8. выполнить теоретическую оценку затрачиваемой реализацией каждого алгоритма памяти;
9. оценить временную эффективность алгоритмов или их реализаций в лучшем и худшем случае в терминах числа сравнений искомой величины с другими величинами;
10. представить результаты сравнительного анализа в формате таблицы;
11. сделать выводы из сравнительного анализа реализации разработанных алгоритмов решения одной и той же задачи, по критериям временной и ёмкостной эффективности, а также дополнительных требований, налагаемых алгоритмом.

1 Аналитическая часть

Постановка задачи: дана последовательность целых чисел $a = (a_0, a_1, \dots, a_n)$, где n — длина последовательности, и искомое значение T . Требуется найти такой индекс i последовательности чисел a , что $a_i = T$.

1.1 Метод полного перебора (Линейный поиск)

Алгоритм последовательного перебора выполняет один линейный проход по элементам несортированной последовательности:

- инициализировать счётчик $i \leftarrow 0$;
- для каждого элемента a_i :
 - если $a_i = T$, вернуть i ;
 - выполнить $i \leftarrow i + 1$;
- вернуть -1 .

Алгоритм оперирует базовыми операциями индексации и проверкой равенства значений.

1.2 Рекурсивный бинарный поиск

Алгоритм. Рекурсивная функция поиска принимает на вход отсортированную по возрастанию последовательность чисел A , искомое значение T , а также левую (l) и правую (r) границы поиска:

- если $l > r$, вернуть -1 ;
- иначе:
 - инициализировать $m \leftarrow \lfloor \frac{l+r}{2} \rfloor$;
 - если $T = a_m$, вернуть m ;
 - иначе если $T < a_m$, выполнить рекурсивный поиск для левой половины (вызвать функцию с границами $l, m - 1$);
 - иначе выполнить рекурсивный поиск для правой половины (вызвать функцию с границами $m + 1, r$).

1.3 Итеративный бинарный поиск №1 (с двумя сравнениями)

Классический итеративный подход сравнивает ключ T со средним элементом a_m отсортированного массива на каждой итерации. Если они равны, поиск завершается, в противном случае область сужается влево или вправо:

- инициализировать $l \leftarrow 0, r \leftarrow n - 1$;
- пока $l \leq r$:
 - инициализировать $m \leftarrow \lfloor \frac{l+r}{2} \rfloor$;
 - если $T = a_m$, вернуть m ;
 - иначе если $T < a_m$, выполнить $r \leftarrow m - 1$;
 - иначе выполнить $l \leftarrow m + 1$;
- вернуть -1 .

1.4 Итеративный бинарный поиск №2 (с одним сравнением)

Альтернативный итеративный алгоритм откладывает проверку на точное равенство до завершения основного цикла сужения диапазона:

- инициализировать $l \leftarrow 0, r \leftarrow n - 1$;
- пока $l < r$:
 - инициализировать $m \leftarrow l + \lfloor \frac{r-l}{2} \rfloor$;
 - если $a_m > T$, выполнить $r \leftarrow m - 1$;
 - иначе выполнить $l \leftarrow m$;
- если $l \leq r$ и $a_l = T$, вернуть l ;
- вернуть -1 .

Вывод

Описана задача поиска элемента в последовательности чисел, разработаны 4 алгоритма (линейный, рекурсивный и два итеративных бинарных поиска) для решения поставленной задачи.

2 Конструкторская часть

В разделе рассматриваются рекурсивный и нерекурсивные алгоритмы решения задачи, разработанные алгоритмы описываются схемами алгоритмов, также приводятся требования к программному обеспечению.

2.1 Схема алгоритма полного перебора

На рисунке 1 показан алгоритм полного перебора.

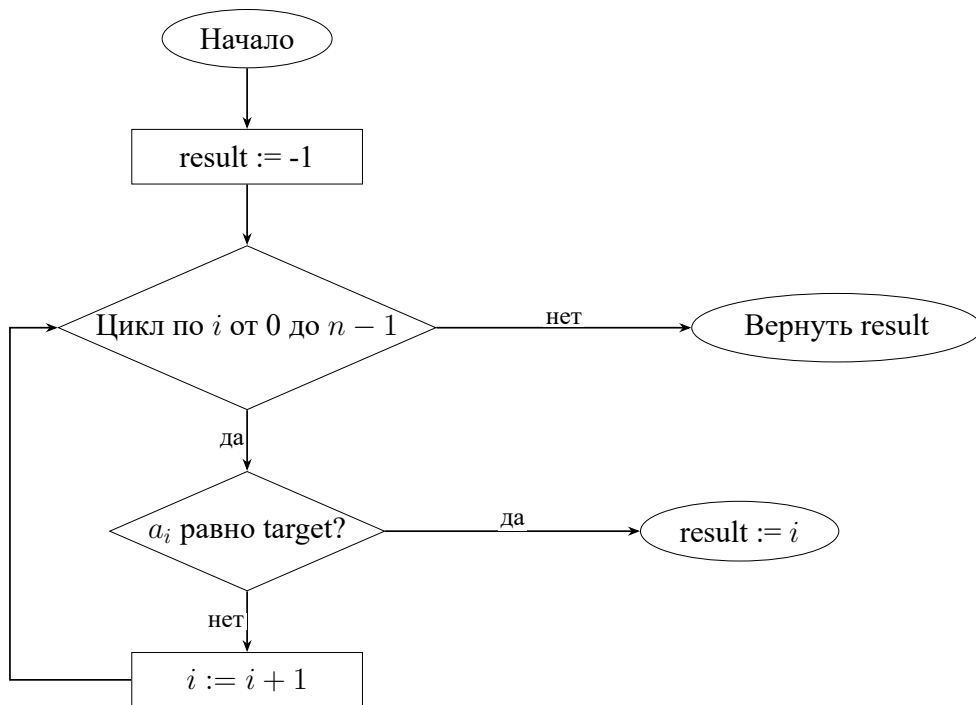


Рисунок 1 – Алгоритм полного перебора

2.2 Схема алгоритма бинарного поиска с двумя сравнениями

На рисунке 2 алгоритм бинарного поиска с двумя сравнениями.

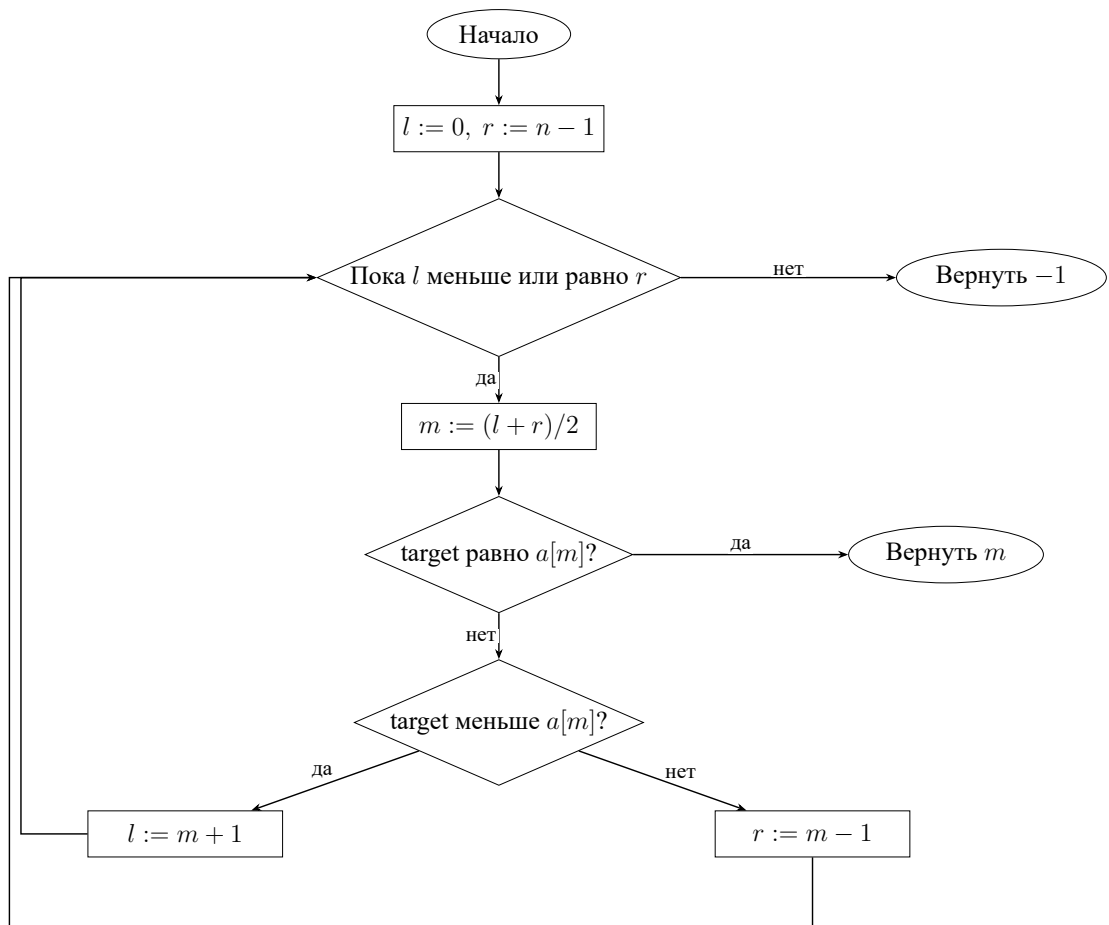


Рисунок 2 – Алгоритм бинарного поиска с двумя сравнениями

2.3 Схема алгоритма бинарного поиска с одним сравнением

На рисунке 3 алгоритм бинарного поиска с одним сравнением.

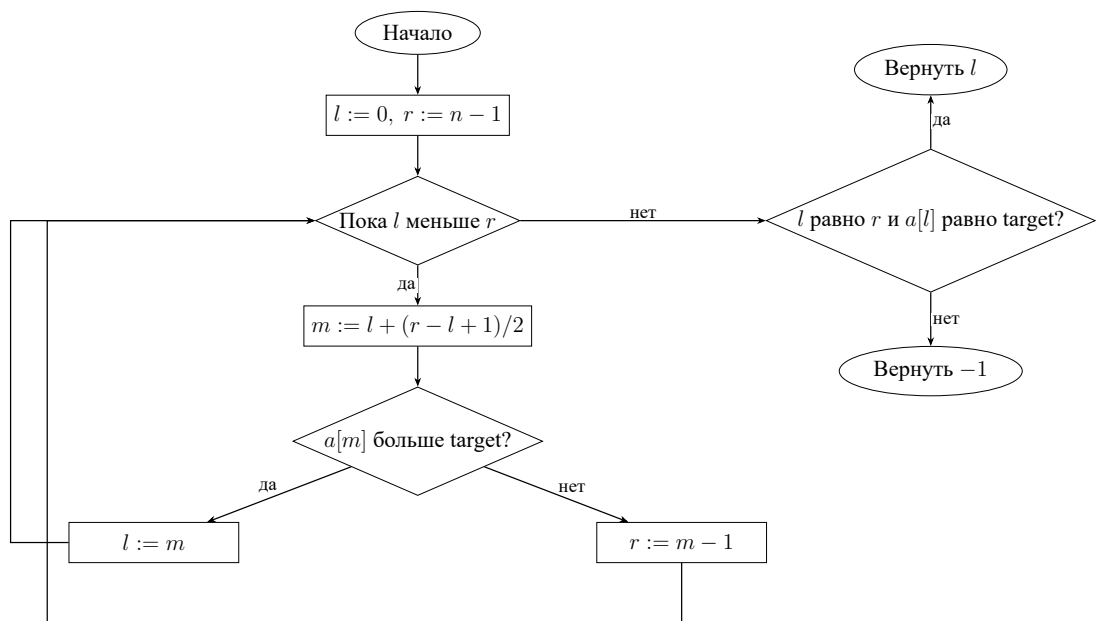


Рисунок 3 – Алгоритм бинарного поиска с одним сравнением

2.4 Схема алгоритма рекурсивного бинарного поиска

На рисунке 4 алгоритм рекурсивного бинарного поиска.

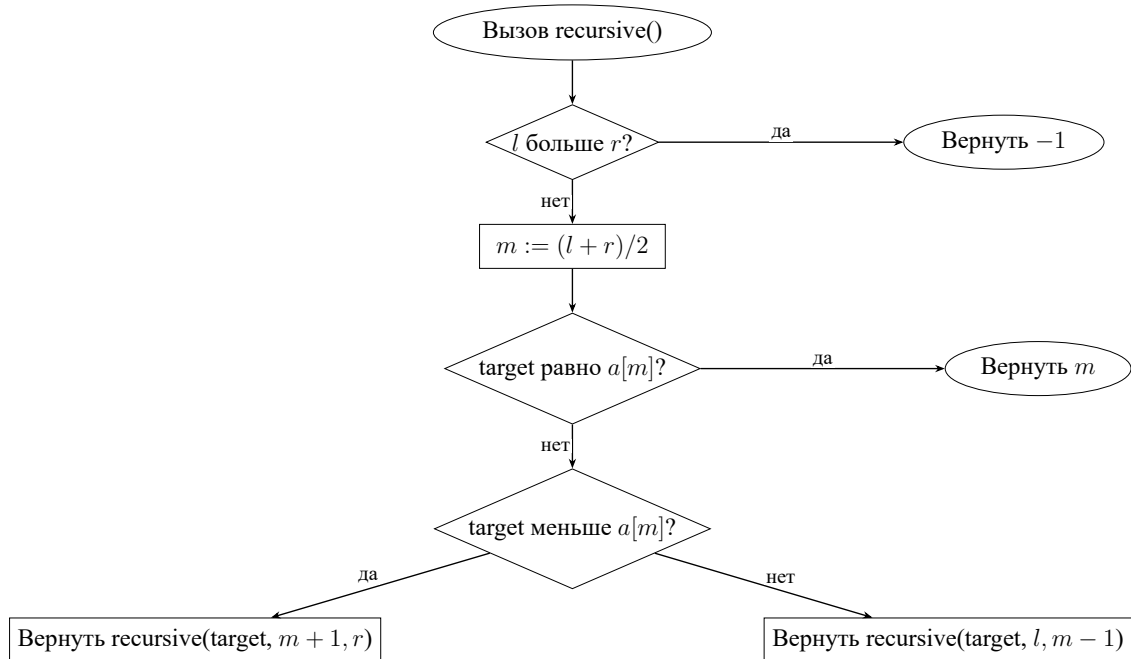


Рисунок 4 – Алгоритм рекурсивного бинарного поиска

2.5 Требования к программному обеспечению

К программному обеспечению предъявляются следующие требования:

1. выбранный язык программирования имеет статическую типизацию (C++);
2. при входе пользователь вводит n — длину последовательности чисел;
3. формируется последовательность чисел из случайных элементов с возможностью пересоздания;
4. создаётся и сортируется копия последовательности чисел;
5. циклически выводится интерактивное меню, посредством которого пользователь может ввести искомое значение и выбрать конкретный алгоритм поиска, а также замерить время и число сравнений.

Вывод

Разработаны рекурсивный и нерекурсивные алгоритмы решения задачи, алгоритмы структурированы для описания схемами, сформулированы четкие требования к программному обеспечению.

3 Технологическая часть

В разделе описываются средства реализации алгоритмов, реализовываются разработанные алгоритмы и выполняется тестирование реализации алгоритмов.

3.1 Средства реализации

Программные реализации алгоритмов выполнены на языке C++. Используемая среда разработки Visual Studio Code. Для точного замера времени использовалась библиотека `<chrono>`, а для генерации данных `<random>`.

3.2 Реализация алгоритмов

В листингах 3.1–3.4 приведены модифицированные реализации алгоритмов поиска элемента в последовательности чисел, которые помимо необходимых для вычисления ответа параметров принимают указатель на переменную `comparison_count`, необходимую для подсчёта количества сравнений.

Листинг 1 – Реализация алгоритма полного перебора

```
int linearSearch(const std::vector<int>& arr, int target, long long&
    comparisonCount) {
    comparisonCount = 0;
    for (size_t i = 0; i < arr.size(); ++i) {
        comparisonCount++;
        if (arr[i] == target) {
            return static_cast<int>(i);
        }
    }
    return -1;
}
```

Листинг 2 – Реализация нерекурсивного алгоритма бинарного поиска с двумя сравнениями

```
int binarySearchIterativeTwoCmp(const std::vector<int>& arr, int target,
    long long& comparisonCount) {
    comparisonCount = 0;
    int left = 0;
    int right = static_cast<int>(arr.size()) - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        comparisonCount++;
        if (arr[mid] == target) {
            return mid;
        }
        comparisonCount++;
        if (arr[mid] < target) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }
    return -1;
}
```

Листинг 3 – Реализация алгоритма нерекурсивного алгоритма бинарного поиска с одним сравнением

```
int binarySearchIterativeOneCmp(const std::vector<int>& arr, int target,
    long long& comparisonCount) {
    comparisonCount = 0;
    int left = 0;
    int right = static_cast<int>(arr.size()) - 1;
    while (left < right) {
        int mid = left + (right - left + 1) / 2;
        comparisonCount++;
        if (arr[mid] > target) {
            right = mid - 1;
        } else {
            left = mid;
        }
    }
    if (left <= right && !arr.empty()) {
        comparisonCount++;
        if (arr[left] == target) {
            return left;
        }
    }
    return -1;
}
```

Листинг 4 – Реализация рекурсивного алгоритма бинарного поиска

```
int binarySearchRecursiveHelper(const std::vector<int>& arr, int target,
    int left, int right, long long& comparisonCount) {
    if (left > right) {
        return -1;
    }
    int mid = left + (right - left) / 2;
    comparisonCount++;
    if (arr[mid] == target) {
        return mid;
    }
    comparisonCount++;
    if (arr[mid] < target) {
        return binarySearchRecursiveHelper(arr, target, mid + 1, right,
        comparisonCount);
    } else {
        return binarySearchRecursiveHelper(arr, target, left, mid - 1,
        comparisonCount);
    }
}

int binarySearchRecursive(const std::vector<int>& arr, int target, long
    long& comparisonCount) {
    comparisonCount = 0;
    return binarySearchRecursiveHelper(arr, target, 0, static_cast<int>(arr
    .size()) - 1, comparisonCount);
}
```

4 Модульное тестирование

Для проверки корректности работы программы проведено модульное тестирование. Результаты представлены в таблицах 3.1–3.3.

Таблица 1 – Модульные тесты алгоритма полного перебора

Сценарий	Входные данные	Эталонный результат	Полученный результат
Искомый элемент находится на первой позиции последовательности	$A = \{9, 8, 7, 6, 5\}, T = 9$	0	0
Искомый элемент нет	$A = \{9, 8, 7, 6, 5\}, T = 1$	–1	–1
Искомый элемент находится на последней позиции последовательности	$A = \{9, 8, 7, 6, 5\}, T = 5$	4	4
Пустая последовательность	$A = \{\}, T = 5$	–1	–1
Единственный элемент последовательности совпадает с искомым значением	$A = \{0\}, T = 0$	0	0
Единственный элемент последовательности не совпадает с искомым значением	$A = \{0\}, T = 3$	–1	–1

Все реализованные алгоритмы бинарного поиска имеют одинаковые сценарии тестирования. В таблице не указаны названия алгоритмов, так как в результате тестирования полученные результаты всех алгоритмов бинарного поиска совпали.

Таблица 2 – Модульные тесты алгоритмов бинарного поиска

Сценарий	Входные данные	Эталонный результат	Полученный результат
Искомый элемент находится в середине последовательности нечётной длины	$A = \{0, 1, 2, 3, 4, 5, 6, 7, 8\}, T = 4$	4	4
Искомый элемент находится на первой позиции последовательности нечётной длины	$A = \{0, 1, 2, 3, 4, 5, 6, 7, 8\}, T = 0$	0	0
Искомый элемент находится на последней позиции последовательности нечётной длины	$A = \{0, 1, 2, 3, 4, 5, 6, 7, 8\}, T = 8$	8	8
Искомый элемент нет в последовательности нечётной длины	$A = \{0, 1, 2, 3, 4, 5, 6, 7, 8\}, T = -1$	-1	-1
Искомый элемент находится в середине последовательности чётной длины	$A = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}, T = 4$	4	4
Искомый элемент находится на первой позиции последовательности чётной длины	$A = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}, T = 0$	0	0
Искомый элемент находится на последней позиции последовательности чётной длины	$A = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}, T = 8$	8	8
Искомый элемент нет в последовательности чётной длины	$A = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}, T = -1$	-1	-1

Таблица 3 – Модульные тесты алгоритмов бинарного поиска, часть 2

Сценарий	Входные данные	Эталонный результат	Полученный результат
Пустая последовательность	$A = \{\}, T = 0$	-1	-1
Единственный элемент последовательности совпадает с искомым значением	$A = \{0\}, T = 0$	0	0
Единственный элемент последовательности не совпадает с искомым значением	$A = \{0\}, T = -1$	-1	-1
Неотсортированная последовательность	$A = \{5, 4, 3, 2, 1, 0\}, T = 0$	Ошибка	Ошибка

Вывод

Описаны средства реализации алгоритмов, реализованы разработанные алгоритмы, выполнено модульное тестирование реализации алгоритмов. Все проверенные сценарии дали ожидаемый результат.

5 Исследовательская часть

5.1 Теоретическая оценка затрачиваемой памяти

В алгоритме полного перебора и нерекурсивных бинарных поисках не задействуются дополнительные динамические структуры данных, используются только скалярные переменные, объем которых не зависит от входного размера n . Следовательно, оценка затрачиваемой памяти для них составляет $O(1)$.

В алгоритме рекурсивного бинарного поиска на каждый уровень вложенности вызова расходуется память системного стека под локальные переменные. В лучшем случае (элемент найден на первой итерации) глубина рекурсии равна 1, затраты памяти $O(1)$. В худшем случае (элемент отсутствует) глубина вызовов достигает $\log_2 n$, что дает оценку затрачиваемой памяти $O(\log n)$.

5.2 Оценка вычислительной сложности алгоритмов как функции от числа сравнений

Пусть количество элементов последовательности равно n . Для работы алгоритмов бинарного поиска входной массив должен быть предварительно отсортирован по возрастанию.

5.2.1. Метод полного перебора

Худшим случаем является отсутствие искомого элемента в последовательности (либо его нахождение в самом конце), тогда сложность по числу сравнений составляет $O(n)$. Лучший случай — нахождение элемента на первой позиции ($O(1)$).

5.2.2. Итеративный бинарный поиск с двумя сравнениями

Лучший случай — элемент находится ровно в центре массива на первой итерации ($O(1)$). Худший случай — элемент отсутствует, сложность составляет $O(\log n)$.

5.2.3. Итеративный бинарный поиск с одним сравнением

Алгоритм совершает фиксированное число шагов сужения диапазона, а проверка равенства вынесена за цикл. Вычислительная сложность в общем случае составляет $O(\log n)$.

5.2.4. Рекурсивный бинарный поиск

Аналогично итеративным аналогам, сложность по числу ключевых сравнений составляет $O(1)$ в лучшем случае и $O(\log n)$ в худшем случае.

5.3 Сравнительный анализ алгоритмов

В таблице 4.1 приведены результаты анализа алгоритмов по количеству затрачиваемой памяти и вычислительной сложности как функции от числа сравнений.

Таблица 4 – Результаты сравнительного анализа алгоритмов

Алгоритм	Оценка вычислительной сложности алгоритмов как функции от числа сравнений		Требования к алгоритму	Оценка затрачиваемой памяти
	Л.с.	Х.с.		
Полный перебор	$O(1)$	$O(n)$	—	$O(1)$
Итеративный бинарный поиск №1	$O(1)$	$O(\log n)$	Последовательность должна быть отсортирована	$O(1)$
Итеративный бинарный поиск №2	$O(\log n)$	$O(\log n)$	Последовательность должна быть отсортирована	$O(1)$
Рекурсивный бинарный поиск	$O(1)$	$O(\log n)$	Последовательность должна быть отсортирована	Л.с.: $O(1)$, х.с.: $O(\log n)$

Вывод

Выполнена теоретическая оценка памяти и временной эффективности в терминах числа сравнений. Установлено, что алгоритмы бинарного поиска

ка значительно эффективнее полного перебора при больших n . Рекурсивный вариант уступает итеративным по потреблению памяти в худшем случае из-за стека вызовов, а на все бинарные поиски накладывается дополнительное требование предварительной сортировки.

ЗАКЛЮЧЕНИЕ

Выполнен сравнительный анализ ресурсной эффективности алгоритмов поиска элемента в массиве чисел.

Решены следующие задачи:

1. описана решаемая задача;
2. разработаны рекурсивный и нерекурсивные алгоритмы решения задачи;
3. разработанные алгоритмы описаны схемами алгоритмов;
4. описаны требования к программному обеспечению;
5. описаны средства реализации алгоритмов;
6. реализованы разработанные алгоритмы;
7. выполнено тестирование реализации алгоритмов;
8. выполнена теоретическая оценка затрачиваемой реализацией каждого алгоритма памяти;
9. оценена временная эффективность реализаций алгоритмов в лучшем и худшем случае в терминах числа сравнений искомой величины с другими величинами;
10. результаты сравнительного анализа представлены в формате таблицы;
11. сделаны выводы из сравнительного анализа реализации разработанных алгоритмов решения задачи, по критериям временной и ёмкостной эффективности, а также дополнительных требований, налагаемых алгоритмом.

По результатам исследования подтверждено, что бинарные алгоритмы обладают логарифмической сложностью $O(\log n)$ и превосходят линейный перебор $O(n)$. Нерекурсивные алгоритмы демонстрируют более стабильное и экономичное потребление памяти по сравнению с рекурсивным подходом.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Кормен Т., Лейзерсон Ч., Ривест Р., Штайн К. Алгоритмы: построение и анализ. 3-е изд. М.: Вильямс, 2022. 1296 с.
2. Левитин А. В. Алгоритмы: введение в разработку и анализ. Москва: Издательский дом «Вильямс», 2006.
3. Documentation – The Go Programming Language [Электронный ресурс]. Режим доступа: <https://go.dev/doc> (дата обращения: 10.09.2026).